1. Write the python program to solve 8-Puzzle problem

```python
from collections import deque

def solve(start, goal):
    queue = deque([(start, [])])
    visited = {start}

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path + [state]

        zero = state.index(0)
        row, col = divmod(zero, 3)

        for dr, dc in moves:
            nr, nc = row + dr, col + dc

            if 0 <= nr < 3 and 0 <= nc < 3:
                new_zero = nr * 3 + nc
                new_state = list(state)

                new_state[zero], new_state[new_zero] = \
                    new_state[new_zero], new_state[zero]

                new_state = tuple(new_state)

                if new_state not in visited:
                    visited.add(new_state)
                    queue.append((new_state, path + [state]))

start = (1, 2, 3,
         4, 0, 6,
         7, 5, 8)

goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

solution = solve(start, goal)

for state in solution:
    print(state)
```

```
(1, 2, 3, 4, 0, 6, 7, 5, 8)
(1, 2, 3, 4, 5, 6, 7, 0, 8)
(1, 2, 3, 4, 5, 6, 7, 8, 0)
```

2. Write the python program to solve 8-Queen problem

```python
def solve(row, cols, diag1, diag2, board):
    if row == 8:
        print(board)
        return True

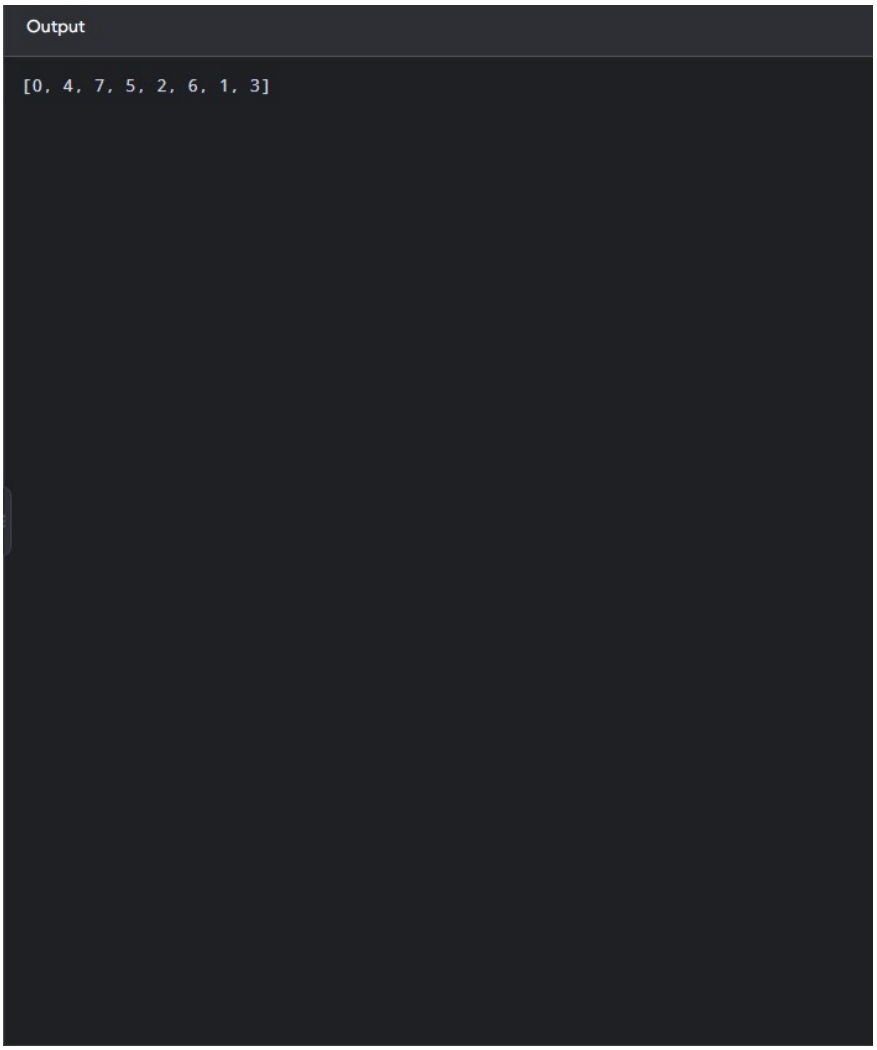
    for col in range(8):
        if col not in cols and row - col not in diag1 and row + col not in diag2:
            board.append(col)

            if solve(row + 1,
                     cols | {col},
                     diag1 | {row - col},
                     diag2 | {row + col},
                     board):
                return True

            board.pop()

    return False


solve(0, set(), set(), set(), [])
```



3. Write the python program for Water Jug Problem

```python
from collections import deque

def water_jug(cap_x, cap_y, target):
    start = (0, 0)
    visited = {start}
    queue = deque([(start, [start])])

    while queue:
        (x, y), path = queue.popleft()

        if x == target or y == target:
            return path

        moves = [
            (cap_x, y),  # fill X
            (x, cap_y),  # fill Y
            (0, y),      # empty X
            (x, 0),      # empty Y
            # pour X -> Y
            (x - min(x, cap_y - y), y + min(x, cap_y - y)),
            # pour Y -> X
            (x + min(y, cap_x - x), y - min(y, cap_x - x)),
        ]

        for state in moves:
            if state not in visited:
                visited.add(state)
                queue.append((state, path + [state]))

    return None

x_cap, y_cap, goal = 4, 3, 2
solution = water_jug(x_cap, y_cap, goal)

if solution:
    print("Steps (4L jug, 3L jug):")
```

```
print("Steps (4L jug, 3L jug):")
for step in solution:
    print(step)
else:
    print("No solution possible")
```

```
Output

Steps (4L jug, 3L jug):
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
```

4. Write the python program for Cript-Arithmetic problem

main.py

Visualize

Run ▶

```python
from itertools import permutations

def solve_cryptarithmetic(word1, word2, result):
    letters = sorted(set(word1 + word2 + result))
    if len(letters) > 10:
        return None

    first_letters = {word1[0], word2[0], result[0]}

    def to_num(word, mapping):
        return int("".join(str(mapping[ch]) for ch in word))

    for perm in permutations(range(10), len(letters)):
        mapping = dict(zip(letters, perm))

        # no leading zeros
        if any(mapping[ch] == 0 for ch in first_letters):
            continue

        if to_num(word1, mapping) + to_num(word2, mapping) == to_num(result, mapping):
            return mapping
    return None

w1, w2, res = "SEND", "MORE", "MONEY"
solution = solve_cryptarithmetic(w1, w2, res)

if solution:
    print("Solution mapping:")
    for letter in sorted(solution):
        print(f"{letter} = {solution[letter]}")
    print()
    n1 = "".join(str(solution[c]) for c in w1)
    n2 = "".join(str(solution[c]) for c in w2)
    nr = "".join(str(solution[c]) for c in res)
    print(f"{n1} + {n2} = {nr}")
else:
    print("No solution found")
```

Output

```
Solution mapping:
D = 7
E = 5
M = 1
N = 6
O = 0
R = 8
S = 9
Y = 2

9567 + 1085 = 10652
```

5. Write the python program for Missionaries Cannibal problem

main.py Visualize Run ▶

```python
from collections import deque

def is_valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False
    # missionaries must not be outnumbered on either bank
    if m != 0 and m < c:
        return False
    if (3 - m) != 0 and (3 - m) < (3 - c):
        return False
    return True

def solve():
    start = (3, 3, 1)   # (missionaries left, cannibals left, boat: 1=left, 0=right)
    goal = (0, 0, 0)
    moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]

    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        (m, c, b), path = queue.popleft()
        if (m, c, b) == goal:
            return path

```

```
        for dm, dc in moves:
            if b == 1:
                nm, nc = m - dm, c - dc
            else:
                nm, nc = m + dm, c + dc
            state = (nm, nc, 1 - b)
            if is_valid(nm, nc) and state not in visited:
                visited.add(state)
                queue.append((state, path + [state]))
    return None

solution = solve()
if solution:
    print("(Missionaries left, Cannibals left, Boat side)")
    for m, c, b in solution:
        print((m, c, "Left" if b == 1 else "Right"))
    print("Crossings:", len(solution) - 1)
else:
    print("No solution")
```

Output

```
(Missionaries left, Cannibals left, Boat side)
(3, 3, 'Left')
(3, 1, 'Right')
(3, 2, 'Left')
(3, 0, 'Right')
(3, 1, 'Left')
(1, 1, 'Right')
(2, 2, 'Left')
(0, 2, 'Right')
(0, 3, 'Left')
(0, 1, 'Right')
(1, 1, 'Left')
(0, 0, 'Right')
Crossings: 11
```